■ Mushroom Business Management & E-Commerce Platform (Mushroom ERP)

Document 06: Role-Based Access Control (RBAC) & Security Architecture

Project Name: SporeERP
Document Code: DOC-06-RBAC-SEC
Security Framework: Least-Privilege Role-Based Access Control (NIST RBAC Standard)
Session Framework: NextAuth.js v5 (Stateless Signed JWT + Server-Side Permissions Validation)

---

1. Security Principles & Architecture Overview

SporeERP enforces strict **Least-Privilege Role-Based Access Control (RBAC)** to prevent operational errors, protect sensitive financial records, and isolate customer data:

```
                                    AUTHENTICATED PRINCIPAL
                                              ■
                                ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
                                ■                            ■
                                ▼                            ▼
                       [CUSTOMER PORTAL]             [INTERNAL STAFF ERP]
                       (Role: CUSTOMER)              (Assigned Staff Role)
                                ■                            ■
                                ■                            ▼
                                ■             ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
                                ■             ■    Role Permission Check    ■
                                ■             ■   (Next.js Edge Middleware)■
                                ■             ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
                                ■                            ■
                                ▼                            ▼
                         [Public Access]              [Route Granted / 403]
```

---

2. System Roles Definition & Operational Scope

Role Identifier	Display Name	Operational Scope & Access Boundary
`FOUNDER_ADMIN`	Co-Founder / Super Admin	**Unrestricted Access:** Master P&L, employee salaries, pricing overrides, role assignment, audit log inspection, system business settings.
`FARM_MANAGER`	Operations Manager	**Full Operations:** Manages production batches, harvest logs, inventory adjustments, supplier purchases, customer orders, and operational reports. (Restricted from Founder salaries and system settings).
`PRODUCTION_STAFF`	Agronomy & Cultivation Staff	**Farm Floor Only:** View/update batches, log daily bed statuses, record room humidity/temp, input daily harvest weights, log contamination loss. (No financial or customer data access).
`INVENTORY_STAFF`	Warehouse & Stock Keeper	**Stock & Raw Materials:** Manage raw material balances, log packaging stock in/out, view inventory transactions, report wastage and spoilage.
`OPERATIONS_STAFF`	Fulfillment & Dispatch Agent	**Order Processing:** View order queue, update packing statuses, print packing slips, assign delivery drivers, update dispatch timestamps.

Role Identifier	Display Name	Operational Scope & Access Boundary
`CUSTOMER`	Retail / B2B Customer	**Storefront & Account:** Browse products, manage personal cart, place orders, complete payments, track live order status, manage delivery addresses.

---

3. Comprehensive Role-to-Permission Matrix

The following matrix defines the granular permissions configured in `role_permissions`:

Module	Atomic Permission Code	Customer	Production Staff	Inventory Staff	Operations Staff	Farm Manager	Founder Admin
Auth & Users	`users:view`	■	■	■	■	■■ Read	■ Full
	`users:manage_`	■	■	■	■	■	■ Full
Catalog	`catalog:product`	■■ Read	■■ Read	■■ Read	■■ Read	■■ Read	■ Full
	`catalog:produ`	■	■	■	■	■ Write	■ Full
	`catalog:pricing:override`	■	■	■	■	■	■ Full
Orders	`orders:create`	■ Yes	■	■	■	■	■ Full
	`orders:view_all`	■	■	■■ Read	■■ Read	■■ Read	■ Full
	`orders:update`	■	■	■	■ Write	■ Write	■ Full
	`orders:cancel_refund`	■	■	■	■	■ Write	■ Full
Inventory	`inventory:vie`	■	■■ Read	■■ Read	■■ Read	■■ Read	■ Full
	`inventory:adjust_stock`	■	■	■ Write	■	■ Write	■ Full
	`inventory:wri`	■	■	■ Write	■	■ Write	■ Full
Production	`production:batch:create`	■	■ Write	■	■	■ Write	■ Full
	`production:ba`	■	■ Write	■	■	■ Write	■ Full
	`production:beds:manage`	■	■ Write	■	■	■ Write	■ Full
Harvest	`harvest:log_e`	■	■ Write	■■ Read	■	■ Write	■ Full
	`harvest:quality_grade`	■	■ Write	■■ Read	■	■ Write	■ Full
Suppliers	`suppliers:man`	■	■	■■ Read	■	■ Write	■ Full
	`purchases:create`	■	■	■ Write	■	■ Write	■ Full
Expenses	`expenses:rec`	■	■	■	■	■ Write	■ Full
	`expenses:view_all`	■	■	■	■	■■ Read	■ Full
Finance	`finance:pnl:`	■	■	■	■	■	■ Full
	`finance:tax_export`	■	■	■	■	■	■ Full
Audit	`audit:logs:i`	■	■	■	■	■	■ Full

---

4. Next.js Edge Middleware Implementation

The route protection guard intercepts every HTTP request before execution:

```
// middleware.ts
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';
import { getToken } from 'next-auth/jwt';

export async function middleware(req: NextRequest) {
 const { pathname } = req.nextUrl;

 // 1. Allow public storefront & auth endpoints
 if (pathname.startsWith('/api/auth') || pathname.startsWith('/_next') || pathname === '/' || pathname.startsWith('/p
   return NextResponse.next();
 }

 // 2. Extract verified JWT token
 const token = await getToken({ req, secret: process.env.NEXTAUTH_SECRET });

 // 3. Unauthenticated redirect to login
 if (!token) {
   const loginUrl = new URL('/login', req.url);
   loginUrl.searchParams.set('callbackUrl', encodeURIComponent(pathname));
   return NextResponse.redirect(loginUrl);
 }

 // 4. Admin portal protection
 if (pathname.startsWith('/admin')) {
   const userRole = token.role as string;
   const allowedStaffRoles = ['FOUNDER_ADMIN', 'FARM_MANAGER', 'PRODUCTION_STAFF', 'INVENTORY_STAFF', 'OPERATIONS_STA

   if (!allowedStaffRoles.includes(userRole)) {
     return NextResponse.redirect(new URL('/unauthorized', req.url));
   }

   // 5. Restrict Financial & Settings to Founders
   if ((pathname.startsWith('/admin/reports') || pathname.startsWith('/admin/settings')) && userRole !== 'FOUNDER_ADMI
     return NextResponse.redirect(new URL('/admin/dashboard?error=restricted', req.url));
   }
 }

 return NextResponse.next();
}

export const config = {
 matcher: ['/admin/:path*', '/checkout/:path*', '/account/:path*', '/api/v1/:path*'],
};
```

---

5. Audit Logging Architecture

To ensure total accountability, all state-altering mutations (price edits, inventory write-offs, order cancellations, batch completions) are recorded in the `audit_logs` table via a dedicated `AuditService`:

```
// services/audit.service.ts
export class AuditService {
 static async logEvent(params: {
   actorId: string;
   action: string;
   entityType: string;
   entityId: string;
   oldState?: any;
   newState?: any;
   ipAddress?: string;
 }) {
   await prisma.auditLog.create({
     data: {
       actorId: params.actorId,
       action: params.action,
       entityType: params.entityType,
       entityId: params.entityId,
       oldState: params.oldState ? JSON.stringify(params.oldState) : null,
       newState: params.newState ? JSON.stringify(params.newState) : null,
       ipAddress: params.ipAddress || null,
     },
   });
```

```
  }
}

```